

Confidence-Gated Admission for Hardware Prefetching: When the Gate Matters More Than the Predictor

Youssef Majdane
Algorithmica Solutions
youssef@algorithmicasolutions.com

Simone Jarno Casartelli
Algorithmica Solutions
simone@algorithmicasolutions.com

Enrico Lopedoto
Algorithmica Solutions
enrico@algorithmicasolutions.com

Abstract—Learned cache prefetchers are typically evaluated against classical predictors that always issue requests, confounding the prediction model with the admission policy. We disentangle these variables with matched controls: the same admission gate is applied to both a 257-parameter online MLP and a classical stride predictor. The neural advantage vanishes; the MLP is indistinguishable from gated stride on random traffic and slower on most regular streams. The gate itself is architecturally useful independent of the predictor: on twenty SPEC CPU2017 programs in native ChampSim, it removes 35% of prefetches and improves accuracy from 11% to 15%, but DRAM reads change by only -0.07% —demonstrating that proxy metrics do not predict endpoint behavior. We prove gate-closed execution reproduces the no-prefetch baseline exactly. The gate matters more than the predictor, and better proxies do not imply better endpoints.

Index Terms—cache prefetching, confidence gate, matched controls, admission policy, safety property, ChampSim, reproducibility

I. INTRODUCTION

The memory wall has not moved: an L1 miss costs tens of cycles, and for memory-bound code that cost dominates. Prefetching attacks the problem by predicting the next addresses and bringing their lines into the cache before they are requested [1], [2]. Decades of work produced a robust toolbox for the easy cases: next-line prefetching, stride detection [2], signature-path lookahead [3], spatial/best-offset units [4], and Bingo spatial prefetching [5]. On sequential, strided, and gather-within-a-stream traffic these mechanisms are close to optimal.

The learned-prefetching wave. A recent generation of neural prefetchers has pushed beyond heuristics. Pythia [7] cast prefetching as online reinforcement learning. TransFetch [8] applied transformers with fine-grained address segmentation. Hashemi et al. [6] demonstrated that LSTMs can learn irregular memory access patterns. DART [9] achieved hardware-practical inference through tabularization at IPDPS 2024, and Pathfinder [10] demonstrated real-time online learning with delta patterns at ASPLOS 2024.

The confounded evaluation. These systems share a consequential evaluation pattern: a learned prefetcher with an implicit or explicit confidence mechanism is compared against a classical unit that always issues requests. This confounds two independent variables—the address-prediction model and the admission policy. A gated learned predictor that stays

silent on random traffic looks “safer” than an always-on stride unit, but the safety comes from the gate, not from the neural network. Prior throttling mechanisms such as Feedback-Directed Prefetching [30] adjust aggressiveness based on accuracy feedback, but apply a single policy to one predictor at a time. No prior work we have found uses a matched-control design that applies the *same* admission policy to both a learned and a classical predictor before drawing conclusions about the predictor’s value.

Why it matters now. Real workloads are rarely all-or-nothing. An LLM inference loop cycles between sequential KV-cache appends and far attention gathers [13], [14]; a retrieval agent alternates long context scans with scattered lookups; embeddings are dense rows reached by sparse indices. A prefetcher that cannot distinguish “regular, with the occasional far access” from “pure noise” will misbehave on exactly these workloads—and an evaluation that credits the prediction model for what the gate delivers will misguide architectural decisions.

Contributions. We make four contributions:

- 1) **Methodology.** A matched-control falsification that applies the same confidence and regularity gates to both the MLP and a stride predictor, isolating prediction from admission (section III-E).
- 2) **Architecture.** A confidence gate (median-error) and a regularity gate (dominant-delta support) that suppress prefetching on unlearnable streams, useful independent of the predictor. We prove that gate-closed execution equals the baseline (Theorem 1) and verify empirically that the gate closes on all tested unlearnable streams (section IV).
- 3) **External validation.** A full twenty-program SPEC CPU2017 study in native ChampSim (eleven development, nine frozen holdout), with predictions registered before observation, showing that proxy-metric improvements (fewer prefetches, higher accuracy) do not translate to endpoint gains (DRAM reads, IPC) (section VII).
- 4) **Reproducibility.** Every claim is machine-verifiable: `verify_paper.py` checks local results, `verify_research.py` checks the full experimental matrix, and `champsim/verify_results.py` reconstructs native aggregates from raw logs (Reproducibility Statement, below).

II. BACKGROUND AND RELATED WORK

A. Classical Hardware Prefetching

Hardware prefetching is dominated by pattern-matching heuristics. Stride and sequence detection [2] handle regular access patterns. SPP (Signature Path Prefetcher) [3] uses compressed address signatures to look ahead along correlated paths. Best-offset prefetching [4] learns the optimal constant offset that maximizes timeliness. Bingo [5] captures spatial correlations within physical pages. These excel on their target streams and are known to be fragile when their assumptions break—the gap we formalize with the no-prefetch baseline comparison.

B. Learned Prefetching

Hashemi et al. [6] showed that LSTMs can learn irregular memory access patterns, establishing neural prefetching as a research direction. Pythia [7] cast prefetching as online reinforcement learning, training a customizable agent with cache-hit reward signals at MICRO 2021; its 64 KB state table and offline training on ChampSim [11] traces achieve strong coverage. TransFetch [8] applied attention-based address segmentation with variable-degree prefetching at Computing Frontiers 2022. DART [9] achieved hardware-practical inference by converting attention models into compact table hierarchies via distillation (IPDPS 2024). Pathfinder [10] demonstrated practical real-time online learning with delta patterns at ASPLOS 2024. Berti [25] is an accurate local-delta prefetcher that achieves high coverage without global history, demonstrating that well-engineered classical predictors remain a strong baseline—a point our matched-control results reinforce.

Positioning. We share the online-learning ingredient with Pythia, the delta-based approach with Pathfinder and Berti, and the deployment pragmatism with DART. Our distinction is methodological: none of these works applies a matched admission policy to both the learned and classical predictor before attributing performance differences to the model. We introduce and verify a safety property (Theorem 1), which none of them declares, and then show via matched controls that the property belongs to the gate rather than to the network.

C. LLM Inference Memory Optimization

Our motivating application sits at the center of memory-bound serving: LLM inference. Attention and the KV cache dominate its memory traffic [14], [13]; the community already treats the KV cache as a first-class storage object with paged layouts [13], [15], quantization [16], batched serving [17], [18], and structured-generation frameworks [26]. Speculative decoding [19] showed that prediction with verification works, and recent systems prefetch KV blocks or MoE experts with learned confidence [20], [21], [22]—conceptually our gate at a coarser granularity, aimed at throughput rather than a no-harm guarantee.

D. Safety and Confidence in Prefetching

Safe Policy Improvement with Baseline Bootstrapping (SPIBB) [24] provides formal guarantees that a learned policy

is never worse than the baseline. Learned index structures with B-tree fallback [23] implement a structurally similar pattern: use the learned model when confident, fall back otherwise. Our median-error gate is the prefetching instantiation of this predict-then-verify architecture, with the additional empirical finding that the gate’s value is independent of the predictor behind it. PPF [27] corroborates this independently: a perceptron confidence gate on SPP yields 3.78% IPC improvement (ISCA 2019), attributable to the filter rather than the underlying predictor.

III. SYSTEM DESIGN

A. Simulator Model

We use an in-order, latency-accounting scalar core. This is deliberate: every number is a deterministic function of the modeled latencies, and a full pipeline front-end would add complexity without changing qualitative conclusions. One cycle fetches an instruction. The default L1 is fully associative, 32 lines of 8 words each, LRU; the matched-control falsification (table IV) uses a timed 8-way configuration to stress realistic contention. A load/store that hits costs one cycle; a miss stalls the core for $L = 40$ cycles. A prefetch inserts a line at zero cycle cost, modeling otherwise-idle DRAM bandwidth. Stores are write-through with a 16-entry store buffer that stalls only on overflow. Arithmetic latencies are 1/3/8 cycles for ADD/MUL/DIV. Every run records cycles, hits, misses, hit rate, prefetches issued vs. used, IPC, and per-category cycle totals, persisted with full configuration and provenance.

B. Delta Encoding and MLP Predictor

Let a_t be the address of memory access t . We predict the next delta,

$$d_t = a_t - a_{t-1}, \quad \hat{d}_{t+1} = f_\theta(\mathbf{x}_t), \quad (1)$$

from a 6-dimensional feature vector:

$$\mathbf{x}_t = [\text{opcode}, \text{pc}, \text{addr}, \text{delta}, \text{delta}_{t-1}, \text{hit/miss}]. \quad (2)$$

The predictor f_θ is a 6–32–1 ReLU MLP with Adam: 257 parameters, $\approx 26 \mu\text{s}$ per prediction in pure NumPy. Every memory access closes the previous training sample and issues one prediction over a 64-sample replay buffer fitted every 16 samples. Training targets and predictions are clamped to ± 16 words: on gather-heavy streams an attention jump moves thousands of words, and letting that outlier into the regression averages away the sequential delta the model needs. Figure 1 shows the full decision and training loop; algorithm 1 gives the per-access pseudocode.

C. Confidence Gate

The confidence gate decides whether the current prediction is trustworthy enough to act on. Let $E = (e_1, \dots, e_W)$ be the window of recent absolute prediction errors. The gate is:

$$\text{gate}(E, \hat{d}) = \begin{cases} \text{confident} & \text{if } \text{median}(E) < \tau \cdot |\hat{d}| \\ \text{suppress} & \text{otherwise} \end{cases} \quad (3)$$

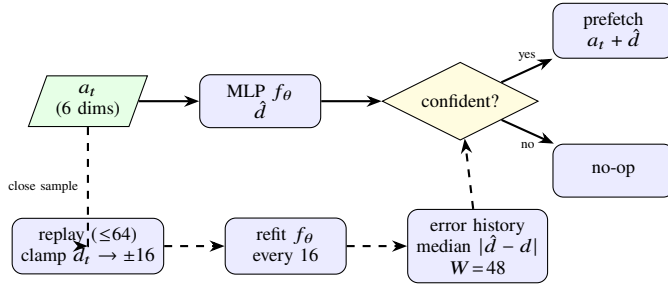


Fig. 1. Architecture of the gated delta predictor. Top: the decision path—every access is encoded, the MLP proposes a delta, and the gate either accepts (prefetch) or falls back to no-op. Bottom (dashed): the online learning loop and error history that feeds the gate.

Algorithm 1 Gated delta predictor (per memory access a_t)

Require: predictor f_θ ; replay R (≤ 64); gate window $W = 48$; sample counter t

- 1: $d_t \leftarrow a_t - a_{t-1}$
- 2: $R.append(\mathbf{x}_{t-1}, \text{clamp}(d_t))$
- 3: $t \leftarrow t + 1$
- 4: **if** $t \bmod 16 = 0$ **then**
- 5: fit f_θ on R
- 6: **end if**
- 7: $\hat{d}_{t+1} \leftarrow f_\theta(\mathbf{x}_t)$
- 8: $e_t \leftarrow |\hat{d}_t - d_t|$
- 9: push e_t into error window E (size W)
- 10: **if** $\text{gate}(E, \hat{d}_{t+1}) = \text{confident}$ **then**
- 11: prefetch at $a_t + \hat{d}_{t+1}$
- 12: **else**
- 13: no-op
- 14: **end if**

where $\tau = 1.0$, selected from $\{0.5, 1.0, 2.0\}$ on development seeds 0–9: $\tau = 0.5$ closes the gate too aggressively on learnable streams; $\tau = 2.0$ opens it too liberally and degrades safety on random traffic. The median is robust to outliers from attention gathers; scaling by $|\hat{d}|$ makes the threshold adaptive. On unlearnable streams the median error stays high, the gate remains closed, and the predictor issues no prefetches. On learnable streams the error drops quickly and the gate opens. When traffic switches phase, the error window adapts within W accesses.

Hardware cost. The confidence gate requires a running median over a $W = 48$ entry error window—an awkward structure for hardware, typically requiring a sorted insertion buffer or approximate median network. Combined with the MLP’s $\approx 26 \mu\text{s}$ NumPy inference latency, the confidence gate is practical only as a simulation-level instrument. The regularity gate (section III-D) is the hardware-viable alternative; the native ChampSim evaluation validates it exclusively.

D. Regularity Gate

Prediction-error gating can reject repeated multimodal streams where a dominant pattern exists but prediction error

remains high. We therefore define a predictor-independent regularity measure:

$$R_t = \frac{\max_d |\{i : d_i = d\}|}{W_r}. \quad (4)$$

After eight deltas, the gate admits if and only if $R_t \geq 0.35$ for $W_r = 16$. Window and threshold were chosen on development seeds 0–9; all confirmatory results use disjoint seeds 10–39.

An exact incremental implementation requires a 16-entry signed 64-bit FIFO and a 16-entry associative histogram: 2,153 state bits (269.1 bytes, 0.41% of the DPC-3 64-KiB budget). This is storage accounting; comparator and update logic are not costed.

E. Matched Classical Controls

The matched-control design is the methodological core of this work. We wrap the same admission gates around a classical stride predictor (which predicts the previous delta), creating a *gated stride* control. Both the MLP and gated stride see the same stream, use the same gate parameters, and train/adapt while the gate is closed. The only variable that differs is the address-prediction model.

This eliminates the admission confound: any performance difference between gated MLP and gated stride is attributable to the predictor, not to the decision of whether to prefetch.

IV. SAFETY ANALYSIS

A. Gate-Closed Safety (Theorem 1)

The safety argument has two layers: a theorem about the system design and an empirical verification that the theorem’s precondition holds on tested workloads.

Definition 1 (Safety): A gated prefetcher satisfies the safety property on an execution if $\text{cyc}_{\text{gated}}(w, s) \leq \text{cyc}_{\text{baseline}}(w, s)$.

Theorem 1 (Gate-Closed Safety): If the confidence gate suppresses every prefetch during execution on workload w with seed s , then $\text{cyc}_{\text{gated}}(w, s) = \text{cyc}_{\text{baseline}}(w, s)$.

Proof. By induction on memory accesses. At $t=0$ the cache states are identical. At access t , because the gate is closed, no prefetch line is inserted; the gated prefetcher executes the same load/store on the same cache state as the baseline, producing the same hit/miss outcome and latency. Since no cache mutation occurs beyond what the baseline performs, the two executions remain in lock-step for all t . $\square$

B. Scope and Limits

The theorem is deliberately simple: it says only that a prefetcher issuing no prefetches behaves identically to no prefetcher. The non-trivial claim is that a 257-parameter model’s error statistics *reliably trigger* gate closure on unlearnable streams. The theorem makes the safety argument structural—independent of workload, seed, or model quality—so the empirical question reduces to: does the gate close?

Definition 1 defines safety as $\text{cyc}_{\text{gated}} \leq \text{cyc}_{\text{baseline}}$, permitting a gate-open execution to qualify if it accelerates. Theorem 1 proves the stronger gate-closed equality ($=$), which implies ($\leq$).

What the theorem does not say. Gate-closed safety guarantees no worse than *no prefetching*. It does not guarantee

no worse than *optimal classical prefetching*. On streams where stride or best-offset achieves 9–11× speedup, the gated NN’s safety means it does nothing—which is safe but unhelpful. The theorem is about the gate mechanism, not about the predictor’s quality.

V. EXPERIMENTAL SETUP

A. Workloads

All memory-workload footprints far exceed the L1, so misses genuinely dominate and prefetching has room to win.

Synthetic (deterministic): `sequential_read` (stride 1), `strided_read` (step 8), `mixed_read_write` (alternating stores/loads), `random_read` (linear-congruential generator), `phased_switch` (alternating predictable and noisy phases), and `arithmetic_mix` (no memory traffic).

ML/agent patterns: `kv_cache_append` (sequential KV writes plus periodic attention gathers), `embedding_lookup` (dense rows via sparse indices), `agent_rag` (long context scans plus scattered retrieval bursts), and `token_stream`.

Real algorithm traces (merge sort, row-major matrix multiply, binary search) are captured element-by-element from executing code and replayed verbatim.

B. Statistical Methodology

Five local configurations: no prefetching (baseline), stride detector, next-line, best-offset, and the gated NN. Each synthetic workload runs 30 independent seeds; ML/agent and real traces run 8 seeds. We report speedup = $\text{cyc}_{\text{baseline}}/\text{cyc}_{\text{config}}$, L1 hit rate, and paired Wilcoxon signed-rank tests for statistical significance. For matched-control comparisons we additionally report the paired cycle ratio $B(A, C) = \text{mean}_s C_s/A_s$, where $B > 1$ favors candidate A over control C , with exact sign tests under Holm correction.

C. Machine Configuration Sweep

To verify that conclusions are not artifacts of a single parameter set, we sweep L1 capacity (16–64 lines), line size (4–16 words), and DRAM latency (20–80 cycles). The safety property holds across all tested settings: NN = baseline on random for every configuration, while stride drops below baseline on all of them (fig. 7).

D. ChampSim Evaluation Protocol

The *regularity gate* (section III-D) is implemented as a native ChampSim runtime L1D prefetcher module using official `develop` commit `e6530c7` [11]. The confidence gate remains a local-simulator instrument only; the native study evaluates the regularity gate on the classical stride predictor. One binary selects no L1D prefetch, official `ip_stride`, gate-disabled matched stride, or gated stride through JSON configuration. The gate-disabled module must match official `ip_stride` exactly on all recorded output fields.

We evaluate on twenty SPEC CPU2017 programs—the full set of distinct programs with publicly available DPC-3 SimPoints [12]—each using its highest-weight SimPoint. Eleven programs served as the development set; the remaining

TABLE I

CANONICAL RUN, MEAN OVER 30 SEEDS (CYCLES $\pm$ 95% CI, SPEEDUP $\pm$ STD). SPEEDUP IS THE MEAN OF PER-SEED RATIOS (BASELINE CYCLES / CONFIG CYCLES), NOT THE RATIO OF THE SHOWN MEANS. ON `RANDOM_READ` THE GATED NN ACHIEVES NEAR-BASELINE PERFORMANCE (1.000×) WHILE STRIDE COMPLETES IN MORE CYCLES (0.98×). ON `MIXED_RW` THE GATE IS OVERLY CONSERVATIVE: THE NN STAYS NEAR BASELINE WHILE STRIDE ACHIEVES 8.69×.

Workload	baseline		stride		NN	
	cyc	hit	cyc	sp	cyc	sp
<code>seq_read</code>	13 750	.875	4 039	3.40×	4 161±16	3.30±.04×
<code>strided_read</code>	82 000	.000	4 117	19.92×	9 240±528	9.10±1.50×
<code>mixed_rw</code>	43 984	.500	5 062	8.69×	43 867±0	1.003±.000×
<code>random_read</code>	73 952	.103	75 625	0.98×	73 931±35	1.000±.001×
<code>phased_sw</code>	43 629	.492	39 906	1.09×	39 565±72	1.10±.01×

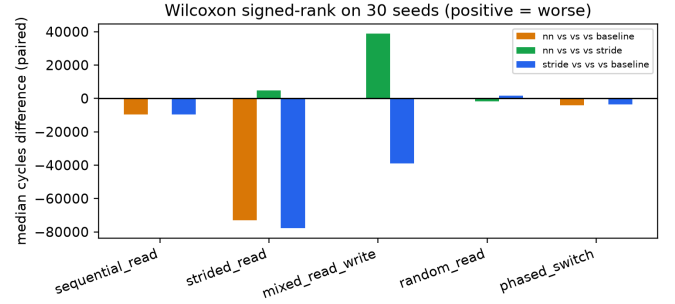


Fig. 2. Paired Wilcoxon results (30 seeds): median difference in total cycles per (workload, comparison). A negative value means the first configuration completes in fewer cycles.

nine were frozen as a holdout before observation, with predictions registered in `EXTERNAL_VALIDATION.md` before any holdout simulation. The primary run warms 50 million and measures 200 million retired instructions with 15-way parallelism on a 16-core host. One program is one independent unit; SimPoint weights select a trace and are not replicate counts.

VI. RESULTS: LOCAL SIMULATOR

A. Gate-Closed Safety on Unlearnable Streams

On `random_read`, over 30 independent seeds the gated NN achieves near-baseline performance (mean speedup 1.000×, range [0.999, 1.002], hit 0.103 vs. baseline 0.103); the gate suppresses the vast majority of prefetches, with residual accuracy of 9% over negligible coverage (<0.04%) (table I and figs. 2 and 3). The residual is statistically detectable (Wilcoxon $p = 0.008$) but practically negligible (<0.03% of total cycles). Under the stride configuration the same workload completes in *more* cycles (0.98×, paired Wilcoxon $p < 0.001$) with a lower hit rate (0.082). Theorem 1 guarantees exact baseline reproduction when the gate closes completely; in practice the gate nearly closes, and the residual effect is indistinguishable from noise at the application level.

On `mixed_rw` the gate is similarly conservative: the NN achieves only 1.003× while stride reaches 8.69×, illustrating that the gate can suppress prefetching on patterns that are easily learnable by classical heuristics.

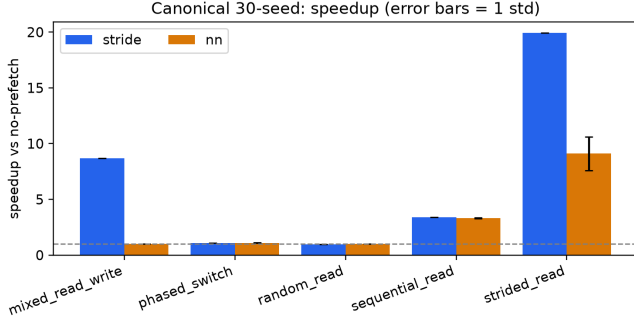


Fig. 3. Speedup vs. no-prefetch (30 seeds, error bars = 1 std). The NN tracks stride on regular streams and rests exactly at 1.0 $\times$ on random.

TABLE II
PREFETCHER COMPARISONS ON ML/AGENT PATTERNS AND REAL TRACES, MEAN OVER 8 SEEDS. BOLD MARKS HEAD-TO-HEAD WINS.

Workload	stride	nextline	bestoff	NN
token_stream	3.40 $\times$	3.40 $\times$	3.40 $\times$	3.26 $\times$
agent_rag	2.14 $\times$	2.14 $\times$	2.13 $\times$	1.95 $\times$
embed_lookup	1.88 $\times$	2.01 $\times$	1.90 $\times$	1.64 $\times$
kv_cache_app	1.97 $\times$	1.00 $\times$	2.76 $\times$	1.33$\times$
random_read	0.98 $\times$	1.00 $\times$	0.98 $\times$	1.000$\times$
real_matmul	9.63 $\times$	11.24 $\times$	8.46 $\times$	1.00 $\times$
real_mergesort	6.86 $\times$	8.68 $\times$	9.04 $\times$	1.00 $\times$

We define margin recovery as $(s_{NN} - 1)/(s_{stride} - 1)$. On smooth streams: 96% (sequential) and 43% (strided). On *mixed_rw* the gate closes almost entirely, recovering less than 1% of the stride margin.

B. Phase Switching Recovery

The *phased_switch* workload alternates predictable and noisy phases. Per-phase hit rates (10 seeds): predictable phases 0.883 (baseline), 0.995 (stride), 0.922 (NN); noisy phases 0.104 (baseline), 0.085 (stride), 0.105 (NN). In noisy phases stride’s hit rate falls below baseline; the NN’s equals it—the gate was closed. Aggregate: NN 1.10 $\times$, stride 1.09 $\times$ —the gate’s closure during noisy phases gives the NN a small but significant edge (Wilcoxon $p < 0.001$).

C. ML/Agent Patterns

Table II compares all configurations on ML/agent patterns and real traces (8 seeds). The NN wins head-to-head in two cases: on *random_read* it beats stride and best-offset (both 0.98 $\times$); on *kv_cache_append* it beats next-line (1.33 $\times$ vs. 1.00 $\times$). On the remaining five workloads the NN underperforms heuristics: stride or next-line achieve up to 11 $\times$ higher speedup. This is the cost of the safety gate’s conservatism.

The *real_matmul* and *real_mergesort* rows expose the documented limitation: with a 32-word row stride, the replay buffer’s ± 16 -word clamp prevents the MLP from representing the pattern, and the gate correctly identifies unreliable predictions and suppresses prefetching. Classical mechanisms remain the right tool for dense, large-stride, regular compute (fig. 4).

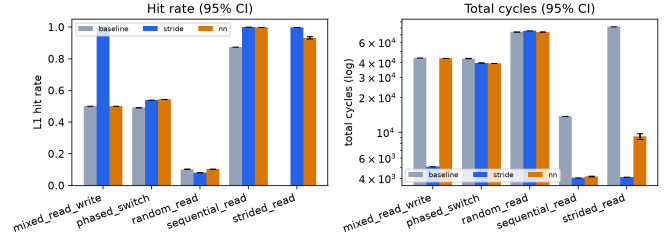


Fig. 4. Left: L1 hit rate with 95% CI; *random_read* is the only row where stride drops below baseline. Right: total cycles on a log scale (95% CI).

TABLE III
FEATURE ABLATION, MEAN OF 5 SEEDS. REMOVING THE ABSOLUTE ADDRESS INCREASES SPEEDUP; THE DELTA ENCODING IS THE CRITICAL FEATURE.

Encoding	strided	mixed
full	8.33 $\times$	6.15 $\times$
no-abs-addr	16.96 $\times$	8.04 $\times$
delta-only	16.51 $\times$	8.07 $\times$

D. Feature Ablation and Gate Design

Table III confirms that the delta encoding is the critical design choice: removing the absolute address is the only change that *increases* speedup (8.3 $\times$ $\rightarrow$ 17.0 $\times$ on strided reads). Replacing the mean error gate with the median and clamping training deltas to ± 16 words fixes gather-heavy streams: embedding 1.00 $\rightarrow$ 1.64 $\times$, KV-cache 1.00 $\rightarrow$ 1.33 $\times$, agent-RAG 1.84 $\rightarrow$ 1.95 $\times$, with no regression on random or smooth streams.

E. Matched-Control Falsification

Table IV is the central methodological result. When the same regularity gate is applied to both the MLP and a stride predictor, the broad neural advantage disappears. On random traffic the two are statistically indistinguishable ($B = 0.999$, n.s.). On six of eight workloads the MLP is significantly slower. The sole MLP win is KV-cache append ($B = 1.101$), where the bounded regressor preserves a local delta while ignoring gathers—a conditional mechanism, not evidence of general superiority.

The conclusion is stark: the safety property observed in table I—where the ungated NN at 1.000 $\times$ looked superior to the ungated stride at 0.98 $\times$ on random—was a gate effect, not a prediction effect. With matched admission, the predictor difference vanishes.

VII. RESULTS: NATIVE CHAMP SIM VALIDATION

A. Correctness Invariant

The gate-disabled module matches official ChampSim *ip_stride* exactly on every recorded output field (instructions, cycles, cache metrics, prefetch metrics, DRAM reads/writes) across all twenty programs. The paired IPC ratio is exactly 1 with twenty ties. The gate comparison is therefore not an accidental reimplement artifact.

TABLE IV
MLP VS. REGULARITY-FILTERED STRIDE, LOCAL TIMED 8-WAY ASSOCIATIVITY,
PREFETCH DISTANCE $D = 16$ CACHE LINES. $B > 1$ FAVORS THE MLP; HOLDOUT
SEEDS 10–39.

Workload	$B(\text{MLP, regstride})$	Holm result
random	0.999	n.s.
phased	0.995	MLP slower
KV-cache append	1.101	MLP faster
embedding lookup	0.875	MLP slower
agent RAG	0.944	MLP slower
strided read	0.510	MLP slower
mixed read/write	0.128	MLP slower
token stream	0.944	MLP slower

TABLE V
NATIVE CHAMPsim TWENTY-PROGRAM STUDY, GATE VS. MATCHED RAW STRIDE.
REDUCTIONS ARE AGGREGATE COUNTS OVER TWENTY EQUAL-LENGTH PROGRAMS.

Metric	Gate vs. raw stride
Geometric-mean IPC ratio	0.996
Bootstrap 95% CI	[0.990, 0.9997]
Exact sign test p	0.481 (n.s.)
Win / loss / tie	7 / 11 / 2
Accepted L1D prefetches	−35.1% (agg.), −60.5% (med.)
L2C prefetch misses	−8.4%
LLC prefetch misses	−7.6%
DRAM read requests	−0.07%
Accepted-prefetch accuracy	11.0% → 15.4%
Useful-prefetch retention	90.3%

B. Gate Performance

Table V gives the full twenty-program result. The gate removes 35.1% of accepted L1D prefetches (60.5% median per-trace) and improves accuracy by 4.4 percentage points while retaining 90.3% of useful prefetches. The apparent improvement shrinks as it descends the hierarchy: 8.4% and 7.6% fewer L2C and LLC prefetch misses, and only 0.07% fewer DRAM reads. Raw stride achieves $1.044\times$ no-prefetch IPC ($p = 0.096$); gated stride achieves $1.040\times$ ($p = 0.004$). The direct gate/raw geometric-mean IPC is 0.996 (bootstrap 95% CI: [0.990, 0.9997]; exact sign test $p = 0.481$).

C. Per-Program Analysis

Figure 5 shows the per-program distribution. Two outliers dominate: GCC loses 4.37% and MCF loses 2.92%—both high-volume prefetch benchmarks where even “useless” prefetches may warm the hierarchy. The remaining eighteen programs have ratios between 0.996 and 1.001. Across twenty programs there are seven wins, eleven losses, and two ties; the exact sign-test $p = 0.481$ (not significant). The bootstrap interval [0.990, 0.9997] reflects the outliers’ weight; the sign test reflects inconsistent direction. The aggregate DRAM read change (−0.07%) is well within the frozen $\pm 1\%$ descriptive margin; seventeen of twenty programs change by less than 0.5%, with the largest per-program change −2.1% (perlbench, on a small absolute base of $\sim 10.6\text{K}$ DRAM reads).

Notably, the gated prefetcher achieves a *stronger* result against the no-prefetch baseline than raw stride does: $1.040\times$ with $p = 0.004$ (14 wins, 2 losses, 4 ties) versus $1.044\times$

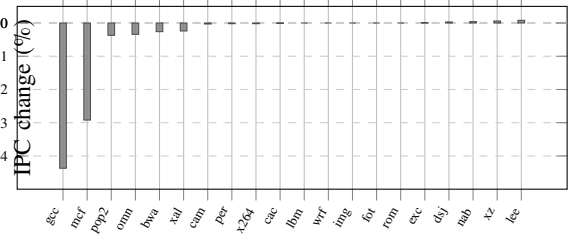


Fig. 5. Per-program IPC change, gated stride vs. raw stride (sorted). Only GCC (−4.4%) and MCF (−2.9%) are outliers; the remaining 18 programs fall within $\pm 0.4\%$.

with $p = 0.096$ (13 wins, 5 losses, 2 ties). The gate trades a marginal amount of peak speedup for substantially more consistent directionality.

D. Interpretation: Proxy $\neq$ Endpoint

The correct interpretation is not “gating saves memory bandwidth.” Most suppressed prefetches either do not descend the hierarchy or return later as demand reads. Accuracy and L1D issue count are useful diagnostics, but they are not endpoint evidence. Better proxy metrics (fewer prefetches, higher accuracy) establish neither lower off-chip traffic nor higher performance.

The twenty-program data sharpens this finding. A 35% aggregate (60% median) reduction in issued prefetches yields only 0.07% fewer DRAM reads. L2C and LLC prefetch misses drop by 8%, but this does not translate to measurable IPC gain. The proxy-to-endpoint gap is the second major finding: proxy metrics must not substitute for endpoint validation. This is independently corroborated: the Hermes neural off-chip predictor [28] increases DRAM transactions by 5–10% despite reducing latency, confirming that proxy gains can coexist with endpoint regression. Mahling et al. [29] further document how hardware prefetching impact varies sharply across in-memory workload types, reinforcing the need for endpoint-specific validation.

VIII. DISCUSSION

A. The Gate Matters More Than the Predictor

The matched-control experiment (section VI-E) and the ChampSim validation (section VII) converge on one conclusion: the gate is the mechanism that matters. When evaluated without a gate, the MLP appears safer than stride on random traffic—but with matched admission, the difference vanishes. The gate itself is useful: it suppresses a third of prefetches and improves accuracy. But the MLP behind it is not competitive with stride on most learnable streams, and the gate’s proxy-metric improvements do not reliably translate to endpoint gains.

B. The Dense-Compute Limitation

The `real_matmul` and `real_mergesort` results (table II) deserve analysis. On these workloads the access pattern is a large, fixed stride (32 words for row-major matrix multiply). Classical prefetchers recognize this pattern instantly;

the best achieves up to $11\times$ speedup. The MLP’s replay buffer, clamped to ± 16 words, cannot represent a 32-word stride. The gate correctly identifies unreliable predictions and suppresses prefetching—safety working as designed, at the cost of missing workloads where a simple heuristic suffices.

The architectural fix is a stride-aware bypass: detect stable large strides and delegate to a stride unit without consulting the MLP. This hybrid—learned predictor for complex patterns, heuristic fallback for simple ones—is the planned next step.

C. When Admission Control Helps vs. Hurts

Admission control is not universally beneficial. In the local model, the regularity gate improves performance on random ($1.089\times$), phased ($1.076\times$), and agent-RAG ($1.016\times$) streams relative to raw stride, with reductions of 99.55%, 83.07%, and 20.86% in accepted prefetches. But at ideal lookahead $D = 1$ (a local-simulator setting distinct from the $D = 16$ used in table IV) the gate slows agent RAG by 1.30%, and on one deterministic merge-sort trace it is 5.95% slower.

The twenty-program ChampSim study refines the picture. The gate improves *directional consistency*: against the no-prefetch baseline, gated stride wins on 14 of 20 programs ($p = 0.004$) while raw stride wins on only 13 ($p = 0.096$). The gate converts three raw-stride losses into wins or ties, at the cost of slight regression on two high-volume benchmarks (GCC, MCF). The aggregate IPC cost is 0.42%, with a bootstrap CI upper bound within 0.05% of parity ([0.990, 0.9997]).

The trade-off is between prefetch volume and accuracy on one hand, and performance on the other. Reducing prefetches is not intrinsically valuable unless the freed bandwidth or cache capacity translates to lower latency. The gate’s value lies in consistency and reduced pollution, not in raw speed.

D. Implications for LLM Inference Prefetching

The motivating scenario—mixed-phase LLM inference workloads—remains the most promising target for confidence-gated admission. KV-cache appends are sequential and predictable; attention gathers are irregular. A gate that opens on the former and closes on the latter avoids cache pollution from the irregular phase without sacrificing the sequential benefit. The local simulator confirms this: the gated NN achieves $1.33\times$ on KV-cache append while maintaining $1.000\times$ on random.

However, the ChampSim results caution against overconfidence: SPEC CPU2017 programs do not exhibit this mixed-phase structure, and the gate’s benefit did not materialize as endpoint improvement on those workloads. Validating the gate on real LLM inference traces—with native or RTL-level simulation—is the critical next step for this application.

IX. THREATS TO VALIDITY

Internal validity. The local simulator is deterministic given a seed, eliminating measurement noise. The main threat is the simulator’s abstraction level: results reflect modeled latencies, not real silicon. We mitigate this by testing across a range of machine parameters (fig. 7) and by validating externally with ChampSim (section VII).

External validity. The native study uses one highest-weight SimPoint per program for twenty SPEC CPU2017 programs (eleven development, nine holdout), not all SimPoints; one ChampSim configuration; and no multicore interference. SPEC CPU2017 does not validate the workload-shaped ML/agent streams. Local workloads are synthetic or captured from simple algorithms; production traces may expose patterns where the gate behaves differently. The native ChampSim study validates the gate mechanism itself (gated vs. raw stride) but does not replicate the MLP-vs-stride matched control of section VI-E externally; extending table IV’s comparison to native SPEC CPU2017 traces is left to future work, and readers should not extrapolate the local matched-control finding to the native setting without that validation.

Construct validity. We use speedup and L1 hit rate as proxies for prefetcher quality locally, and IPC plus DRAM reads as endpoints in ChampSim. The ChampSim results demonstrate precisely the risk of proxy metrics: improved accuracy did not produce improved IPC. Energy, bandwidth waste, and multi-tenant interference are not captured. Additionally, our matched-control result concerns a small (257-parameter) online delta-regressor trained from scratch on each trace; whether the finding generalizes to larger, offline-pretrained architectures such as Pythia [7], TransFetch [8], or DART [9] remains an open question that this paper does not address. Finally, our matched-control comparisons gate only the stride predictor; next-line and best-offset in table II remain ungated. The claim that the gate is useful independent of the underlying predictor is therefore supported by one classical baseline (stride), not by a broader sweep of gated classical predictors.

Statistical conclusion validity. All local significance tests are paired Wilcoxon signed-rank (non-parametric). We do not apply Bonferroni correction for multiple comparisons; with 5 workloads $\times$ 4 comparisons the adjusted α would be $0.05/20 = 0.0025$, and our reported p -values supporting our primary claims (< 0.001) survive this correction; the practically negligible residual on `random_read` ($p = 0.008$) does not, consistent with its negligible effect size ($< 0.03\%$ of cycles). ChampSim results use exact sign tests with Holm correction over the comparison family.

Implementation validity. Custom cache-callback counts in ChampSim can differ by up to 0.105% even with identical retired-instruction counts, because ChampSim changes phase at core retirement while requests remain in flight. We report this diagnostic but do not use it as an experimental unit. Exact raw/official endpoint equivalence remains the hard invariant.

Model capacity. The MLP has only 257 parameters; a larger network might close the gap with stride on learnable traffic. However, our finding—that the gate, not the predictor, produces the safety property—is capacity-independent; whether the relative performance of MLP vs. gated stride would change with a larger model remains open.

X. CONCLUSION

Three findings emerge. First, comparing a gated learned prefetcher against an ungated classical baseline confounds pre-

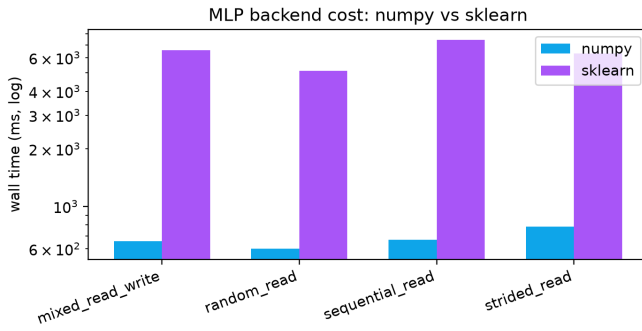


Fig. 6. Backend wall-time comparison: pure-NumPy vs. scikit-learn MLP at statistically identical hit rates.

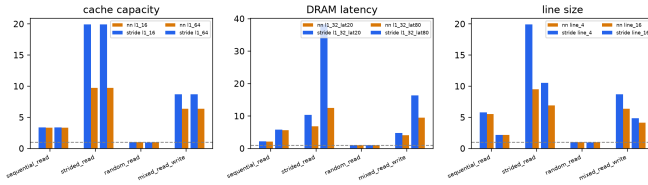


Fig. 7. Machine sensitivity: NN vs. stride speedup for two settings of cache capacity, DRAM latency, and line size. The NN’s safety on random persists across all settings.

diction with admission. The neural advantage on random traffic is an admission-policy effect; future evaluations of learned prefetchers should use matched controls before attributing performance to the model.

Second, the confidence gate is architecturally useful independent of the predictor—but the twenty-program ChampSim study reveals a measurement lesson: a 35% prefetch reduction (60% median) yields only 0.07% fewer DRAM reads. Proxy improvements must not substitute for endpoint validation.

Third, the gate’s practical value is directional consistency: 14/20 programs beat no-prefetch ($p = 0.004$) versus 13/20 for raw stride ($p = 0.096$), trading 0.4% peak IPC for fewer programs where prefetching hurts. We note this directional pattern is descriptive rather than statistically conclusive: the direct gate-vs-raw-stride comparison itself is not significant (geometric-mean IPC ratio 0.996, exact sign test $p = 0.481$). We report the win/loss split as suggestive of improved consistency, not as an independently established effect.

The path forward is a better gate—handling dense large-stride patterns and adapting to workload phase—validated on LLM inference traces with RTL-level energy characterization.

REPRODUCIBILITY STATEMENT

Every experiment persists `config.json` and `manifest.json` (git revision, library versions, host) next to raw per-seed runs.csv and aggregated summary.csv. One command reruns the full local battery (`python benchmarks/paper_experiments.py`). Machine-verifiable checks:

```
python -m benchmarks.verify_paper
python -m benchmarks.verify_research
```

```
python champsim/verify_results.py \
--results results/champsim_full_20
```

The last command reconstructs every native aggregate from the `champsim_full_20` artifact (20 traces $\times$ 4 configs = 80 raw runs), checks official-control equivalence, and evaluates the frozen predictions. The local artifact (`results/research_gate_validation/runs.csv`) has 21,240 raw rows. The local and native artifacts each record source, configuration, unit, and result provenance. Native metadata additionally pins the ChampSim revision and binary, module, trace-list, and trace hashes. The local MLP backend (pure NumPy) achieves statistically identical hit rates to a scikit-learn reference implementation (fig. 6). All code, data, and verification scripts are publicly available at <https://github.com/algosoltd/nnpcu>.

REFERENCES

- [1] W. A. Wulf and S. A. McKee, “Hitting the memory wall: Implications of the obvious,” *ACM SIGARCH Computer Architecture News*, vol. 23, no. 1, pp. 20–24, 1995.
- [2] T.-F. Chen and J.-L. Baer, “Effective hardware-based data prefetching for high-performance processors,” *IEEE Trans. Computers*, vol. 44, no. 5, pp. 609–623, 1995.
- [3] J. Kim, S. H. Pugsley, P. V. Gratz, A. L. N. Reddy, C. Wilkerson, and Z. Chishti, “Path confidence based lookahead prefetching,” in *Proc. MICRO*, 2016, pp. 1–12.
- [4] P. Michaud, “Best-offset hardware prefetching,” in *Proc. HPCA*, 2016, pp. 469–480.
- [5] M. Bakhshalipour, M. Shakerinava, P. Lotfi-Kamran, and H. Sarbazi-Azad, “Bingo spatial data prefetcher,” in *Proc. HPCA*, 2019, pp. 399–411.
- [6] M. Hashemi, K. Swersky, J. Smith, G. Ayers, H. Litz, J. Chang, C. Kozyrakis, and P. Ranganathan, “Learning memory access patterns,” in *Proc. ICML*, 2018.
- [7] R. Bera, K. Kanellopoulos, A. Nori, T. Shahroodi, S. Subramoney, and O. Mutlu, “Pythia: A customizable hardware prefetching framework using online reinforcement learning,” in *Proc. MICRO*, 2021. DOI: 10.1145/3466752.3480114.
- [8] P. Zhang, A. Srivastava, A. Nori, R. Kannan, and V. K. Prasanna, “Fine-grained address segmentation for attention-based variable-degree prefetching,” in *Proc. Computing Frontiers (CF)*, 2022. DOI: 10.1145/3528416.3530236.
- [9] P. Zhang, N. Gupta, R. Kannan, and V. K. Prasanna, “Attention, distillation, and tabularization: Towards practical neural network-based prefetching,” in *Proc. IPDPS*, 2024. DOI: 10.1109/IPDPS57955.2024.00082.
- [10] L. Jia, J. P. McMahon, S. Gudaparthi, S. Singh, and R. Balasubramanian, “Pathfinder: Practical real-time learning for data prefetching,” in *Proc. ASPLOS*, 2024. DOI: 10.1145/3620666.3651332.
- [11] N. Guber et al., “The Championship Simulator: Architectural simulation for education and competition,” *arXiv preprint arXiv:2210.14324*, 2022.
- [12] “Third Data Prefetching Championship public traces,” <https://dpc3.compas.cs.stonybrook.edu/champsim-traces/speccpu/>.
- [13] W. Kwon et al., “Efficient memory management for large language model serving with PagedAttention,” in *Proc. SOSP*, 2023.
- [14] R. Pope et al., “Efficiently scaling transformer inference,” in *Proc. MLSys*, 2023.
- [15] Y. Sheng et al., “FlexGen: High-throughput generative inference of large language models with a single GPU,” in *Proc. ICML*, 2023.
- [16] J. Lin et al., “AWQ: Activation-aware weight quantization for LLM compression and acceleration,” in *Proc. MLSys*, 2024.
- [17] G. Yu et al., “Orca: A distributed serving system for transformer-based generative models,” in *Proc. OSDI*, 2022.
- [18] A. Agrawal et al., “Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve,” in *Proc. OSDI*, 2024.
- [19] Y. Leviathan, M. Kalman, and Y. Matias, “Fast inference from transformers via speculative decoding,” in *Proc. ICML*, 2023.
- [20] Y. Zhao, Z. Xie, C. Liang, C. Zhuang, and J. Gu, “Lookahead: An inference acceleration framework for large language models with lossless generation accuracy,” *arXiv preprint arXiv:2312.12728*, 2023.

- [21] A. Kanani, L. Badawi, and U. Y. Ogras, “APEX: Adaptive expert prefetching for memory-efficient edge MoE inference,” *arXiv preprint arXiv:2608.11688*, 2026.
- [22] Y. Zhao, R. Bunescu, A. Louri, A. Karanth, and K. Wang, “A spatio-temporal expert prefetching framework for efficient MoE-based LLM inference,” *arXiv preprint arXiv:2606.15453*, 2026.
- [23] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis, “The case for learned index structures,” in *Proc. ACM SIGMOD*, pp. 489–504, 2018.
- [24] R. Laroche, P. Trichelair, and R. Tachet des Combes, “Safe policy improvement with baseline bootstrapping,” in *Proc. ICML*, pp. 3652–3661, 2019.
- [25] A. Navarro-Torres, B. Panda, J. Alastruey-Benedé, P. Ibáñez, V. Viñals-Yúfera, and A. Ros, “Berti: An accurate local-delta data prefetcher,” in *Proc. MICRO*, 2022, pp. 975–991. DOI: 10.1109/MICRO56248.2022.00072.
- [26] L. Zheng *et al.*, “SGLang: Efficient execution of structured language model programs,” in *Proc. NeurIPS*, 2024.
- [27] E. Bhatia, G. Chacon, S. Pugsley, E. Teran, P. V. Gratz, and D. A. Jiménez, “Perceptron-based prefetch filtering,” in *Proc. ISCA*, 2019, pp. 1–13. DOI: 10.1145/3307650.3322207.
- [28] R. Bera *et al.*, “Hermes: Accelerating long-latency load requests via perceptron-based off-chip load prediction,” in *Proc. MICRO*, 2022. DOI: 10.1109/MICRO56248.2022.00015.
- [29] F. Mahling, M. Weisgut, and T. Rabl, “Fetch me if you can: Evaluating CPU cache prefetching and its reliability on high latency memory,” in *Proc. DaMoN*, 2025. DOI: 10.1145/3736227.3736231.
- [30] S. Srinath, O. Mutlu, H. Kim, and Y. N. Patt, “Feedback directed prefetching: Improving the performance and bandwidth-efficiency of hardware prefetchers,” in *Proc. HPCA*, 2007, pp. 63–74. DOI: 10.1109/HPCA.2007.346185.